

# PERFORMANCE OPTIMIZATION USING ARTIFICIAL INTELLIGENCE METHODS

Marta Plantykow, Piotr Raczynski, Maciej Machnikowski, Pawel Szymanski

August 2020

# NOTICES AND DISCLAIMERS

Intel technologies' features and benefits depend on system configuration and may require enabled hardware, software or service activation. Performance varies depending on system configuration.

No product or component can be absolutely secure.

Tests document performance of components on a particular test, in specific systems. Differences in hardware, software, or configuration will affect actual performance. For more complete information about performance and benchmark results, visit http://www.intel.com/benchmarks .

Software and workloads used in performance tests may have been optimized for performance only on Intel microprocessors. Performance tests, such as SYSmark and MobileMark, are measured using specific computer systems, components, software, operations and functions. Any change to any of those factors may cause the results to vary. You should consult other information and performance tests to assist you in fully evaluating your contemplated purchases, including the performance of that product when combined with other products. For more complete information visit http://www.intel.com/benchmarks .

Intel Advanced Vector Extensions (Intel AVX) provides higher throughput to certain processor operations. Due to varying processor power characteristics, utilizing AVX instructions may cause a) some parts to operate at less than the rated frequency and b) some parts with Intel® Turbo Boost Technology 2.0 to not achieve any or maximum turbo frequencies. Performance varies depending on hardware, software, and system configuration and you can learn more at http://www.intel.com/go/turbo.

Intel's compilers may or may not optimize to the same degree for non-Intel microprocessors for optimizations that are not unique to Intel microprocessors. These optimizations include SSE2, SSE3, and SSSE3 instruction sets and other optimizations. Intel does not guarantee the availability, functionality, or effectiveness of any optimization on microprocessors not manufactured by Intel. Microprocessor-dependent optimizations in this product are intended for use with Intel microprocessors. Certain optimizations not specific to Intel microarchitecture are reserved for Intel microprocessors. Please refer to the applicable product User and Reference Guides for more information regarding the specific instruction sets covered by this notice.

Cost reduction scenarios described are intended as examples of how a given Intel-based product, in the specified circumstances and configurations, may affect future costs and provide cost savings. Circumstances will vary. Intel does not guarantee any costs or cost reduction.

Intel does not control or audit third-party benchmark data or the web sites referenced in this document. You should visit the referenced web site and confirm whether referenced data are accurate.

© 2020 Intel Corporation. Intel, the Intel logo, and other Intel marks are trademarks of Intel Corporation or its subsidiaries. Other names and brands may be claimed as the property of others.

# Problem statement

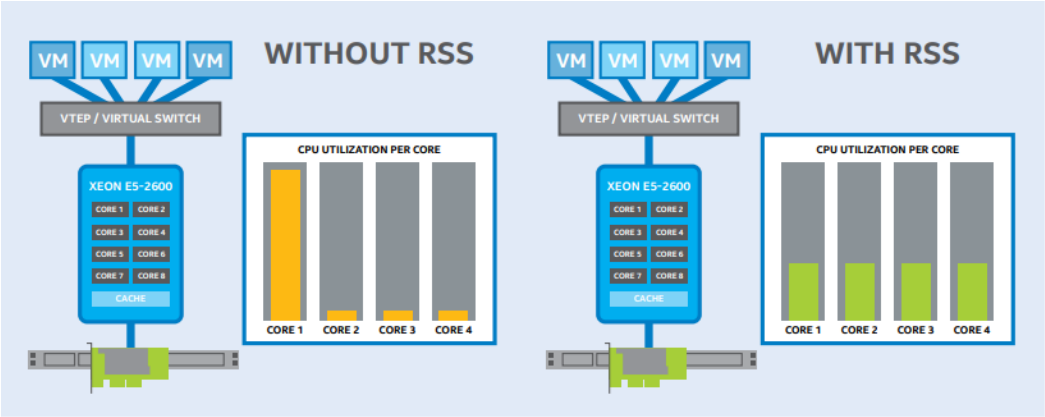
- How to efficiently spread the given UDP/TCP IPv4 traffic evenly amongst the given number of queues with RSS

# Agenda

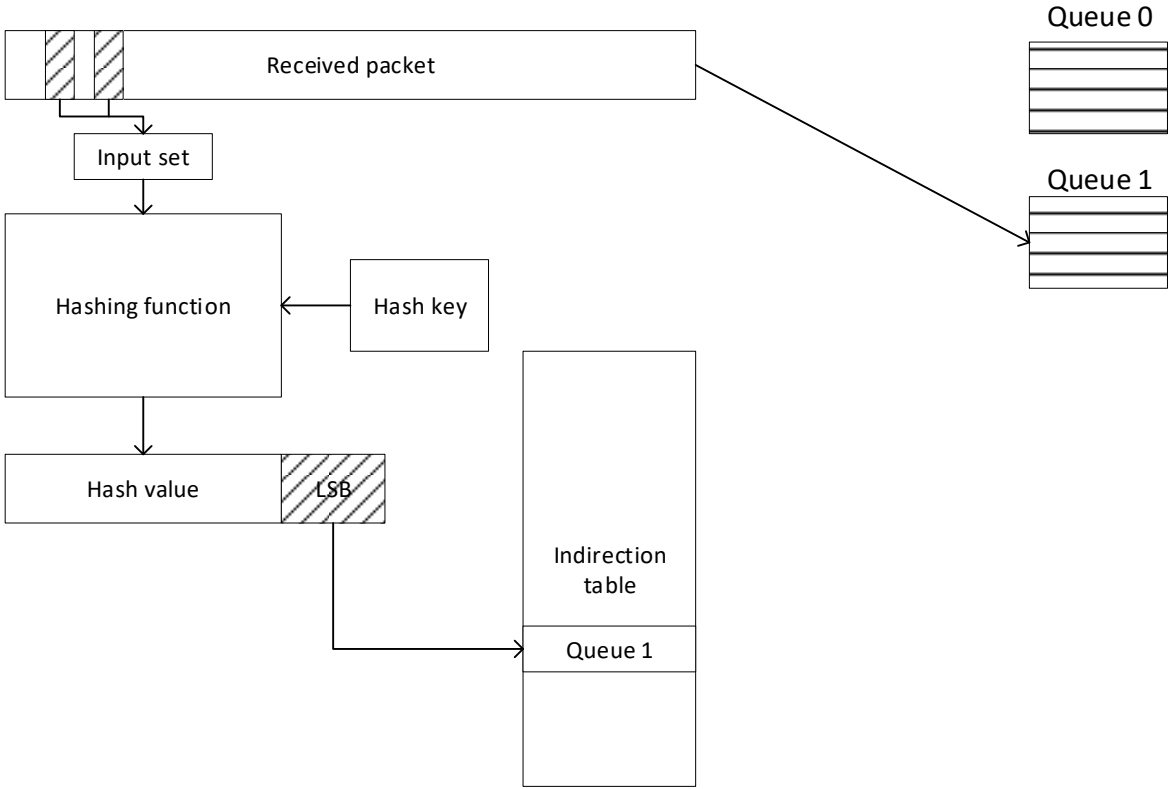
- RSS Introduction
- Problems with RSS
- Key optimization using AI

# What is RSS?

- Method of spreading traffic to many CPUs
- Packets from the same flow are redirected to the same queue
- Some traffic patterns can spread the load unevenly



# What is RSS?


Received packet
Queue 0
Queue 1
Input set
Hashing function
Hash key
Hash value
LSB
Indirection table
Queue 1

# Input set:

## TCP/IPv4: 12 bytes extracted from the incoming packet:

- 4-byte src_ip (161.51.128.175)
- 4-byte dst_ip (192.168.1.1)
- 2-byte src_port (80)
- 2-byte dst_port (80)

| | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0xA1 | 0x33 | 0x80 | 0xAF | 0xC0 | 0xA8 | 0x01 | 0x01 | 0x00 | 0x50 | 0x00 | 0x50 |

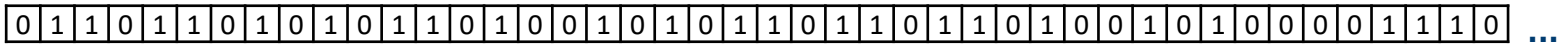
## TCP/IPv6: 36 bytes extracted from the incoming packet:

- 16-byte src_ip (2001:db8:0:0:0:0:2:1)
- 16-byte dst_ip (2002:bc7:0:0:0:0:3:4)
- 2-byte src_port (80)
- 2-byte dst_port (80)

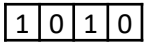
| | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0x20 | 0x01 | 0x0d | 0xb8 | 0x00 | 0x00 | 0x00 | 0x00 | 0x00 | 0x00 | 0x00 | 0x00 | 0x00 | 0x00 | 0x02 | 0x01 | 0x20 | 0x02 |
| 0x0b | 0xc7 | 0x00 | 0x00 | 0x00 | 0x00 | 0x00 | 0x00 | 0x00 | 0x00 | 0x00 | 0x00 | 0x03 | 0x04 | 0x00 | 0x50 | 0x00 | 0x50 |

# Hashing function (Toeplitz)

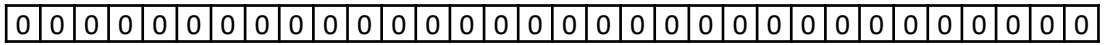
Key [0x6d, 0x5a, 0x56, 0xda, 0x25, 0x5b, 0x0e, 0xc2 ...]:




Input set [0xA...]:

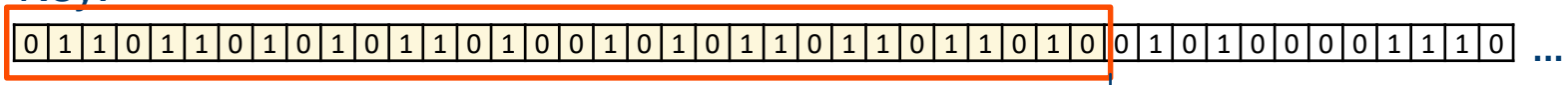



Hashing result:

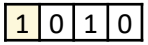


# Hashing function (Toeplitz)

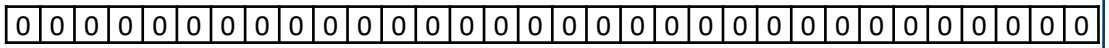
Key:



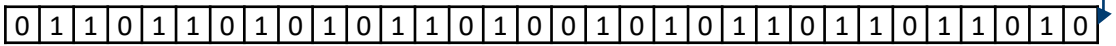

Input set:



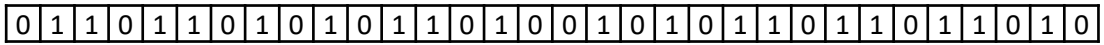

Previous hashing result:




xor

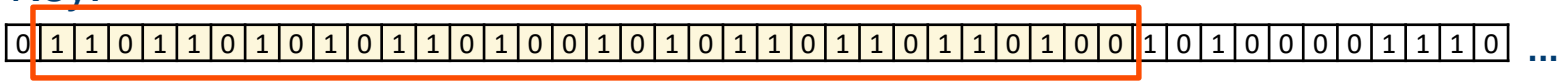



Hashing result:

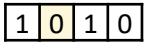


# Hashing function (Toeplitz)

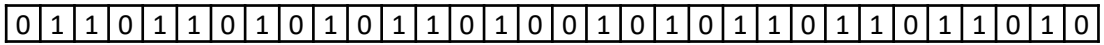
Key:




Input set:

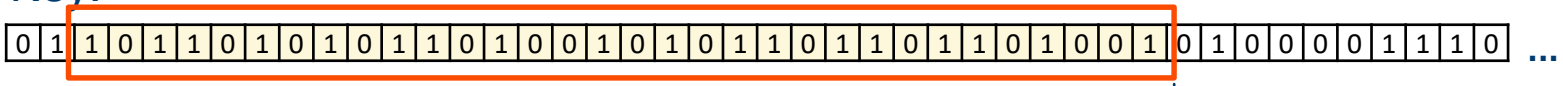



Hashing result:

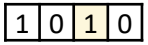


# Hashing function (Toeplitz)

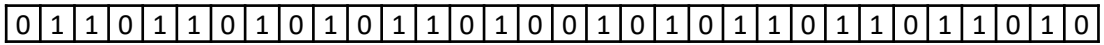
Key:



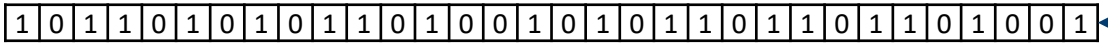

Input set:



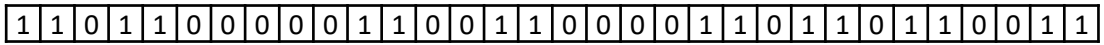

Previous hashing result:




xor

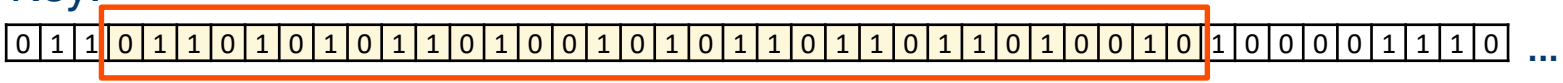



Hashing result:

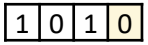


# Hashing function (Toeplitz)

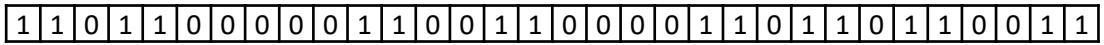
Key:




Input set:

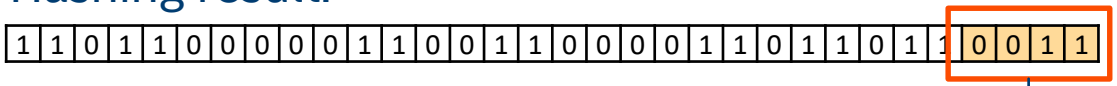



Hashing result:



# Indirection table

## Hashing result:




| idx | qid |
|---|---|
| 0 | 0 |
| 1 | 1 |
| 2 | 0 |
| 3 | 1 |
| 4 | 0 |
| 5 | 1 |
| 6 | 0 |
| 7 | 1 |
| 8 | 0 |
| 9 | 1 |
| 10 | 0 |
| 11 | 1 |
| 12 | 0 |
| 13 | 1 |
| 14 | 0 |
| 15 | 1 |

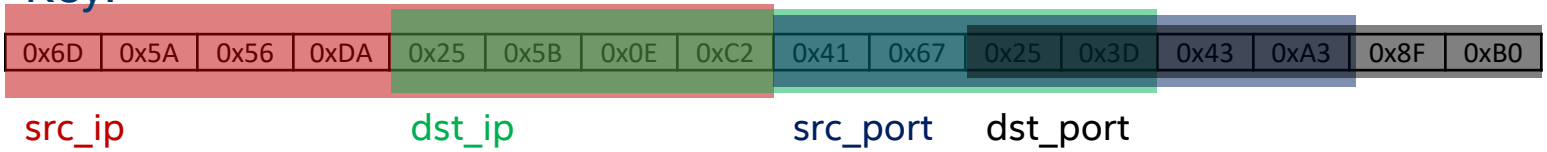
# Key takeaways

- Key parts are associated with the input set sections

Input set:

| 0xA1 | 0x33 | 0x80 | 0xAF | 0xC0 | 0xA8 | 0x01 | 0x01 | 0x00 | 0x50 | 0x00 | 0x50 |
|---|---|---|---|---|---|---|---|---|---|---|---|

Key:




Some parts of the key are used for more than one input set data

# Key takeaways

- Only length(InputSet) +4 bytes of the key are used
  - 12+4=16 bytes for TCP_IPv4
  - 36+4=40 bytes for TCP_IPv6
- It’s not easy to predict where will the packet land

# Problems of RSS

- Lack of entropy in incoming traffic
  - NAT gate (most of the traffic is directed to the same IP address from IP addresses that differ only in the last byte)
  - Web server (traffic has the same dest and src port)
- Bad use of the entropy
  - A lot of 00s in the significant part of the key
- Bunching up traffic with the different hash to the same queue via bad indirection table
- Traffic that ends up with the same hash LSBs (but from different flows) will end up on the same queue and it cannot be changed via indirection table
- RSS can have issues with tunneled traffic

# Ways to improve RSS

Modifying RSS parameters can improve CPU balance but:

- Modifying indirection table
  - Won't help if traffic has the same hash LSB value
  - Breaks in order processing for modified indirection table values
- Modifying RSS key
  - Can help if multiple flows end up with the same hash LSB value
  - But can result in all flows reassociation
  - also breaks in order processing of incoming packets, possibly for all flows
- Input set modification

# Evaluated solutions

- Toeplitz hash key modification

# Evaluated solutions

- Toeplitz hash key modification
  - Random key

# Evaluated solutions

- Toeplitz hash key modification
  - Random key
  - Genetic algorithm

# Evaluated solutions

- Toeplitz hash key modification
  - Random key
  - Genetic algorithms
  - Markov Decision Process

# Proposed solution

- Toeplitz hash key modification
  - Random key
  - Genetic algorithms
  - Markov Decision Process
  - Bayesian Optimization algorithm with the usage of on-policy prediction with approximation

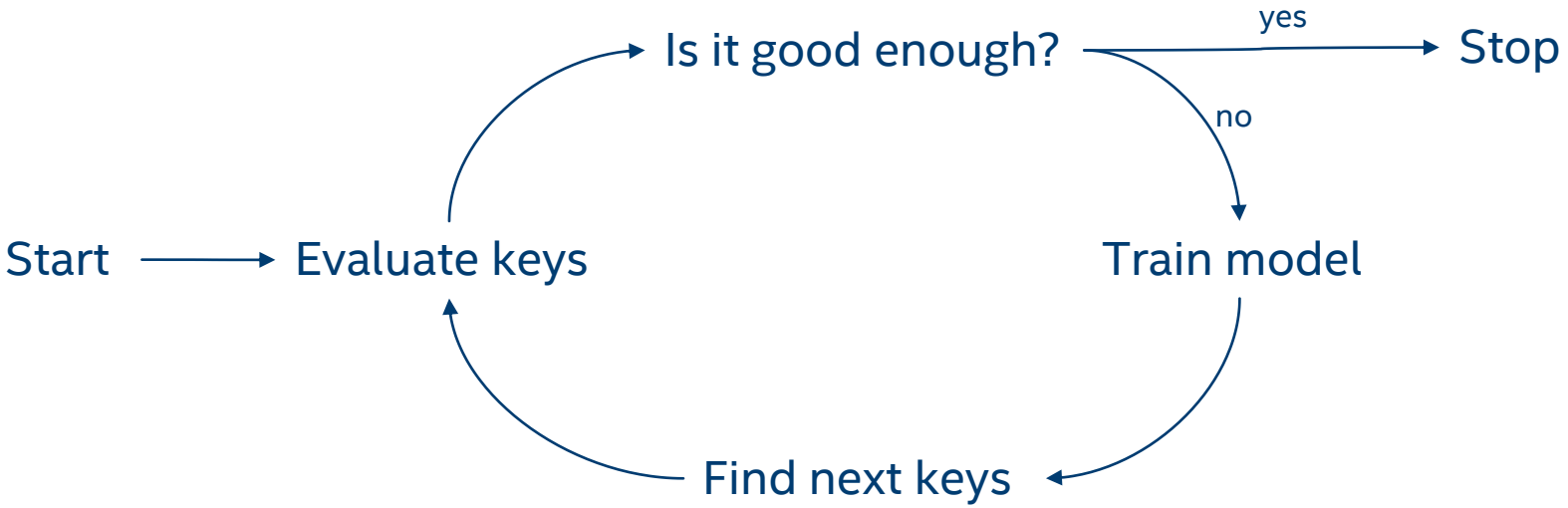
# Setup:

- PCAP file as input
- RSS emulator with key input
- Neural network-based model of an objective function



# Setup:

- PCAP file as input
- RSS emulator with key input
- Neural network-based model of an objective function



# Why AI?

- Formal optimization problem:

$$\min \frac{\sum_{i=0}^{n}(L_i - L_a)^2}{n}$$

- Where:

$$L_a = \frac{\sum_{i=0}^{n}(L_i)^2}{n}$$

- $n$ - number of cores
- $L_i$- current load of a core
- $L_a$ - average core load.

# Why AI?

Artificial intelligence is good at:

- Optimization: Maps, electrical power systems, fuel consumption and CO2 emission on ships, etc.

Local minimum is worse than global, but it is better than a random value.

# Current state

- We are working on a model
  - We don’t have any hard data yet

# Next steps

## Next steps:

- More key generators: random, genetic
- Automation
- Measurements
- IPv6

# THANK YOU!